

Financial Forecasting by Multi-AI Agent System

Complete Phase-by-Phase Work Breakdown (Corrected Build Spec)

B.Tech Capstone Project — step-by-step: every phase, every file, every function, every deliverable.

This is the **build document**: follow it task by task. It merges the detailed engineering steps with the fixes from the technical review, so nothing here leads to a dead end later.

Coverage: worldwide equities (US, India, UK, Japan, EU, HK, AU...), all free data sources.

Golden rule: *build the ruler before the thing you measure*. Evaluation is Phase 2, not Phase 5.

Phase Map at a Glance

Phase	What you build	Result you can show
1	Foundation + data cache + single-LLM baseline	One command returns one real forecast
2	Evaluation harness + leakage-free protocol	Any forecaster gets a scored results table
3	7 agents + debate + orchestrator + <code>--no-debate</code> switch	Type a ticker -> full multi-agent forecast
4	Experiments & ablation	The tables and charts that earn marks
5	Dashboard + report + defence	Web app + thesis + slides

Golden rules that apply to every phase:

- 1 Every data function takes an `as_of date`. No exceptions. (Prevents future-data leakage.)
- 2 Every external call goes **through the cache**. (Prevents rate-limit death on demo day.)
- 3 Every agent returns a **Pydantic model** with **citations**. (Prevents silent hallucination.)
- 4 **Commit to git after every working step.** `pip freeze > requirements.txt` after first good install.

Phase 1 — Foundation & Walking Skeleton

Goal: one command produces one real, schema-valid forecast — thin but complete, end to end.

Task 1.1 — Create the virtual environment

```
cd c:\Users\frien\OneDrive\Desktop\BTP
py -3.11 -m venv venv          # prefer 3.11; 3.13 is very new – see note
venv\Scripts\activate
python --version               # confirm it activated
```

Note on Python version: if `py -3.11` is unavailable, use `py -3 (3.13)`. If `pandas-ta-classic`

or `langgraph` fails to install on 3.13, that is the signal to switch to a 3.11 venv. Decide by the

install result, not by assumption.

Task 1.2 — Install dependencies (pinned)

```
pip install langgraph langchain-google-genai google-genai fastapi uvicorn \
    yfinance pandas pandas-ta-classic pydantic python-dotenv \
    requests feedparser httpx tenacity streamlit
pip freeze > requirements.txt          # DO THIS NOW, not at the end
```

Library	Purpose
langgraph	Multi-agent orchestration — the core framework
langchain-google-genai	Connect LangGraph nodes to Gemini
google-genai	Raw Gemini SDK (tighter control of structured output)
fastapi + uvicorn	Backend API server (Phase 5)
yfinance	Stock prices/financials — unofficial Yahoo scraper
pandas	Data manipulation
pandas-ta-classic	Technical indicators — maintained fork (imports as <code>pandas_ta_classic</code>)
pydantic	Structured agent output schemas
python-dotenv	Load API keys from <code>.env</code>
feedparser	Parse RSS feeds
httpx	Async HTTP client
tenacity	Exponential backoff/retry for 429s (new — not optional)
streamlit	Dashboard (primary UI, Phase 5)

Task 1.3 — Secure ALL API keys (including the ones the old plan forgot)

- **Gemini** -> aistudio.google.com — free tier.
- **FRED** -> fredaccount.stlouisfed.org — free, separate registration. *Needed in Phase 2; get it now.*
- **News** -> **GDELT needs NO key.** (NewsAPI is dropped — its free tier is 1-month archive, 24h delay, localhost-only: useless for backtesting.)

Create `.env` in the project root:

```
GEMINI_API_KEY=your_key_here
FRED_API_KEY=your_key_here
```

Add `.env` to `.gitignore` **before your first commit** so keys never reach GitHub.

Task 1.4 — Project structure

```

BTP/
├── backend/
│   ├── __init__.py      ← REQUIRED (fixes "No module named 'backend'")
│   ├── config.py        ← settings + env loader
│   ├── cache.py         ← Task 2.1 (built first in Phase 2)
│   ├── tools/           ← data tools (Phase 2)
│   │   └── __init__.py
│   ├── agents/          ← 7 agents (Phase 3)
│   │   └── __init__.py
│   ├── graph/           ← LangGraph orchestrator (Phase 3)
│   │   └── __init__.py
│   ├── eval/            ← evaluation harness (Phase 2) – NEW
│   │   └── __init__.py
│   ├── api/             ← FastAPI (Phase 5)
│   │   └── __init__.py
├── frontend/            ← Streamlit/React (Phase 5)
├── tests/
│   └── test_env.py
├── data/                ← cache DB + frozen test set live here
├── .env                  ← never commit
├── .gitignore
├── requirements.txt
└── README.md

```

Every folder that is imported needs an `__init__.py`. This is the single most common Phase-1 bug.

Task 1.5 — config.py

```

import os
from dotenv import load_dotenv
load_dotenv()

class Config:
    GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
    FRED_API_KEY   = os.getenv("FRED_API_KEY")
    # Flash-class model for specialists/debate (cheap, fast); reserve any Pro call for final synthesis
    GEMINI_MODEL   = "gemini-flash-latest"
    CACHE_DB       = "data/cache.db"

```

Task 1.6 — tests/test_env.py (corrected)

```
import sys
print(f"Python: {sys.executable}")

for name, mod in [("yfinance", "yfinance"),
                  ("pandas_ta_classic", "pandas_ta_classic"), # NOT pandas_ta
                  ("google-genai", "google.genai"),
                  ("langgraph", "langgraph")]:
    try:
        __import__(mod)
        print(f"OK {name}")
    except ImportError as e:
        print(f"FAIL {name}: {e}")

# run as: python -m tests.test_env (from project root, so 'backend' is importable)
from backend.config import Config
print("OK config" if Config.GEMINI_API_KEY else "WARN GEMINI_API_KEY not set")
```

Run it: `python -m tests.test_env`

Task 1.7 — The single-LLM baseline (moved up from Phase 5 — critical)

Build `backend/baseline.py`: one Gemini call, given the same data, returns the **same** forecast schema as the full system. **This is the thing the multi-agent system must beat.** It must exist before Phase 2 so the harness can score it.

Task 1.8 — One specialist end-to-end (the pattern)

Build the **Technical** agent fully (data -> indicators -> Gemini -> `TechnicalReport`) as the template the other three will copy in Phase 3.

[OK] Phase 1 Exit Gate

`python -m forecast RELIANCE.NS --as-of 2025-01-15` returns a valid, schema-conformant forecast

from both the baseline and the technical agent — twice in a row, the second run served from

cache with zero network calls. `requirements.txt` committed. Git initialised.

Phase 2 — Data Layer & Leakage-Free Evaluation Harness

Goal: build every data tool *with time-travel built in*, then build the ruler that scores forecasts.

This is the academic core of the project.

Task 2.1 — `cache.py` (BUILD FIRST — not optional)

- SQLite (in `data/cache.db`) sitting in front of **every** external call — prices, news, macro, **and LLM responses**.
- **Two cache classes** (this is the fix the old plan got wrong):

Class	Key	TTL
Historical / point-in-time	(source, ticker, field, as_of)	Permanent — the past never changes
Live quote (demo only)	(source, ticker, field)	15 min
LLM response	sha256(prompt + model)	Permanent

- Why permanent for history: a 15-min TTL forces constant re-fetching during ablation sweeps — triggering the exact 429s the cache exists to prevent.

Task 2.2 — market_data.py (worldwide, time-aware)

```
get_price_history(ticker, start, end, as_of)  # as_of caps the window; never returns rows after it
get_current_price(ticker)                   # live/demo path only
```

- **Worldwide by design:** accept any Yahoo suffix — AAPL, RELIANCE.NS, BP.L, 7203.T, SAP.DE, 0700.HK, BHP.AX. Never hardcode a market.
- **Auto-fallback:** on yfinance HTTP 429 -> retry from **Stooq** (keyless, global, decades of history) via **tenacity** exponential backoff. **Normalise Stooq's columns to match yfinance** — different schema is silent breakage exactly during the outage you built the fallback for.
- Always fetch **>= 2 years** of history regardless of the display window, so SMA-200 never returns NaN.

Task 2.3 — indicators.py

- **pandas_ta_classic** on the price DataFrame: **RSI(14)**, **MACD(12/26/9)**, **SMA 50 & 200**, **Bollinger**, **ATR**.
- Returns values + **plain-English reading** (e.g. RSI 72 -> Overbought).
- Guard against short series (return None + a note, never a wrong number).

Task 2.4 — fundamentals.py (with the point-in-time caveat)

```
get_fundamentals(ticker, as_of)  # P/E, EV/EBITDA, revenue growth, margins, D/E, market cap
get_earnings(ticker, as_of)     # last 4 quarters + surprise %
get_company_info(ticker)        # sector, industry, name, description
```

- **Explicit null handling:** yfinance returns **None** unpredictably for non-US tickers. On a missing field the agent must say *"unavailable"* — **never invent a number**.
- **Documented limitation:** yfinance gives *today's* (restated) financials, not what was public on **as_of**. Mitigate by lagging fundamentals **>= 1** quarter after the reporting date and stating this in the report. (True point-in-time data is institution-priced; out of scope.)

Task 2.5 — news_sentiment.py (GDELT — global, free, time-aware)

```
get_news(ticker, as_of)          # GDELT: filter to published_date <= as_of
score_sentiment(headlines)       # Gemini -> polarity + relevance + confidence + catalysts
```

- **GDELT** replaces NewsAPI: 100+ languages, every country, archive to 1979, free. It rate-limits (429s observed) -> **must go through the cache**.
- **Dedup** by URL + title similarity before scoring, or the same story double-counts and skews polarity.
- GDELT gives metadata/tone, **not full article text** — adequate for sentiment; state as a limitation.
- **Prompt-injection guard:** headlines are untrusted web text. Delimit them clearly and instruct the model to treat them as data, not instructions. Validate output against the schema.

Task 2.6 — macro.py (FRED + regional, time-aware)

```
get_macro_data(as_of)          # FRED: Fed Funds, CPI, 10Y yield, GDP — respecting data vintage
get_sector_performance(sector) # sector index/ETF relative performance
```

- Needs the **FRED key** from Task 1.3. For non-US markets, add the relevant central-bank series (e.g. RBI for India) or note the limitation.
- Returns a **Tailwind / Neutral / Headwind** rating with reasoning.

Task 2.7 — The evaluation harness (backend/eval/) — the part the old plan lacked

- 1 **Determine the model's training cutoff empirically.** Run a **recall probe**: ask Gemini about firm-date outcomes it should not know; verify it cannot recall them. Record the result — it is a figure in your report.
- 1 **Freeze the test set** — tickers, as-of dates, horizon (e.g. 21 trading days) — in `data/testset.json`, **committed to git**. Never edit it after seeing results.
- 1 **Test window must sit AFTER the training cutoff** (the recall probe confirms where that is).
A within-training-window backtest measures memory, not forecasting — it is invalid.
- 1 **Metrics:**
 - Directional accuracy (up/down) — headline number
 - MAE / MAPE on the price target
 - **Confidence calibration** (Brier score / reliability curve) — where multi-agent should win
 - **Cost per forecast** (tokens, seconds, calls) — to answer "is the debate worth it?"
 - A **naive baseline** (momentum persistence) to prove the LLM beats a coin flip
- 1 Harness runs *any* forecaster over the frozen set -> CSV + markdown table.
- 2 **Record baseline results now** — these go in the report regardless of what happens later.

[OK] Phase 2 Exit Gate

`python -m evaluate --system baseline` produces a complete scored table over the frozen set,

reproducibly. The recall-probe result is documented. Every tool works from the terminal, returns

real worldwide data, is cached, and respects `as_of`.

Phase 3 — Multi-Agent System

Goal: the full architecture, built against a harness that already exists.

Task 3.1 — Pydantic schemas (backend/agents/schemas.py)

- **TechnicalReport**: score (-10..+10), trend, support/resistance, indicator values, **citations**
- **FundamentalReport**: valuation score, intrinsic value, health status, **citations**
- **SentimentReport**: polarity (-1..+1), relevance, confidence, catalysts, **article links**
- **MacroReport**: regime rating, key indicators, reasoning

- **DebateArgument**: position (bull/bear), points, target, confidence
- **FinalForecast**: signal (Buy/Hold/Sell), price target range, confidence %, risk report, **citations**

Gemini structured-output gotchas (verified) — test these in Phase 1, not here:

- Gemini rejects schemas using `Field(default=...)`. Use `Optional[...]` and post-process.

- Deeply nested models can be misread as tool calls (Unknown tool name). Keep schemas flat,

prefer prefixed fields over deep object trees.

- Set `propertyOrdering` explicitly (up to ~15% quality gain on complex schemas).

- Count parse failures as a reported metric, not a silent retry.

Task 3.2 — The 4 specialist agents (backend/agents/)

Agent	System-prompt role	Tools (Phase 2)	Output
Technical	"Senior technical analyst — price action, momentum, patterns."	<code>market_data</code> , <code>indicators</code>	<code>TechnicalReport</code>
Fundamental	"Fundamental equity analyst — financial health & valuation."	<code>fundamentals</code>	<code>FundamentalReport</code>
Sentiment	"Market sentiment analyst — mood from news & signals."	<code>news_sentiment</code>	<code>SentimentReport</code>
Macro	"Macroeconomist — broad conditions affecting this stock."	<code>macro</code>	<code>MacroReport</code>

Each agent receives the `as_of date` and must only use data at or before it.

Task 3.3 — Reconciliation gate (new — the anti-hallucination layer)

Before any agent's numbers propagate, **re-verify each number against the cached source data** (Bloomberg's ASKB pattern). A mismatch raises a warning, not a silent pass. Prove it works by injecting a fake number in a test and confirming the gate catches it.

Task 3.4 — Bull vs Bear debate engine

- **Bull**: receives all 4 reports -> strongest upside case, growth catalysts, upside target.
- **Bear**: receives all 4 reports -> downside risks, overvaluation signs, downside floor.
- **Bounded loop**: Bull -> Bear rebuts -> Bull counters -> Bear final. **N=2 rounds** (unbounded debate burns quota fast). Each round logged for the Phase 5 dashboard.

Task 3.5 — Risk Officer / Synthesiser

- Receives all 4 reports + full debate transcript.

- Weighs evidence, applies risk rules -> `FinalForecast` (signal, target range, confidence %, risk report).
- Run at **temperature ~ 0** for reproducibility.

Task 3.6 — LangGraph orchestrator (backend/graph/)

```
from typing import TypedDict, Optional, Annotated
import operator

class AgentState(TypedDict):
    ticker: str
    as_of: str
    # DISTINCT keys per specialist – avoids INVALID_CONCURRENT_GRAPH_UPDATE
    technical_report: Optional[TechnicalReport]
    fundamental_report: Optional[FundamentalReport]
    sentiment_report: Optional[SentimentReport]
    macro_report: Optional[MacroReport]
    debate: Annotated[list, operator.add] # reducer for parallel/accumulated writes
    final: Optional[FinalForecast]
```

- **Parallel** fan-out: Technical, Fundamental, Sentiment, Macro run simultaneously.
- **Sequential** after: specialists -> reconciliation -> debate -> risk officer -> output.
- Add the **SQLite checkpoint** so a crash mid-run resumes, not restarts.
- **Wire the `--no-debate` switch from the start** — routes specialists straight to the synthesiser.

Without it, the central research question cannot be answered.

[OK] Phase 3 Exit Gate

`python main.py AAPL --as-of 2025-06-01` runs all agents -> prints the full forecast with debate.

The same system scores end-to-end on the Phase 2 harness with no manual steps, and the

reconciliation gate catches a deliberately injected fake number.

Phase 4 — Experiments & Ablation

Goal: answer the research question with evidence. **This phase produces the marks.**

- **Primary comparison:** full multi-agent vs single-LLM baseline vs naive baseline.
- **Debate ablation:** full vs `--no-debate` — report accuracy delta **and** cost delta side by side.
- **Specialist ablation:** leave-one-out (drop each specialist). Expect at least one to add nothing —

that is a finding, not a failure.

- **Calibration analysis:** reliability curves per configuration.
- **Repeat runs** (>=3, fixed temperature) to report variance — a single LLM run is an anecdote.
- **Qualitative failure analysis:** 5–10 wrong forecasts, diagnosed. Examiners reward this heavily.

[OK] Phase 4 Exit Gate

Every claim destined for the report is backed by a committed results file, regenerable by one command.

Phase 5 — Dashboard, Report & Defence

Goal: make it demonstrable and defensible. Deliberately last — high demo value, low research value.

Task 5.1 — FastAPI backend

```
POST /api/forecast/{ticker}    -> runs pipeline, returns JSON
WS  /api/stream/{ticker}      -> streams live agent progress ← owns the live UX flow
GET  /api/price/{ticker}      -> chart data
```

- **Decide ownership now:** the WebSocket drives the live agent log (a 30–90 s POST times out on proxies).
- **Security:** per-IP rate limit; explicit CORS (never `allow_origins=["*"]`); `.env` never committed;

no secret behind a `NEXT_PUBLIC_` var; `pip audit` before submission.

Task 5.2 — Dashboard (Streamlit first)

Ticker search · price chart (**TradingView Lightweight Charts** — Recharts has no native candlestick) · **live agent execution log** · **Bull vs Bear split view** · **final forecast card** with confidence + citations. Move to React/Next.js **only if time genuinely remains** — a polished UI earns few marks against a missing ablation study.

Task 5.3 — BTP report

- 1 Introduction & problem statement
- 2 **Literature survey — position against TradingAgents (arXiv:2412.20138)** and cite the lookahead-bias work (arXiv:2512.23847). This is where novelty is defended.
- 1 System design (architecture, agent graph, sequence diagrams)
- 2 Implementation (stack, key algorithms, reconciliation, `as_of` design)
- 3 **Results & evaluation** (leakage-free protocol, all ablations, calibration, cost, failure analysis)
- 4 Conclusion, limitations (point-in-time data, GDELT text), future work

Task 5.4 — Slides + demo

15–20 slides: Problem -> Solution -> Architecture -> Demo -> Results -> Conclusion. **Cache a known-good run** — never let a live API call decide whether your defence succeeds.

[OK] Phase 5 Exit Gate

A clean clone of the repo runs the demo, and every number in the report traces to a committed results file.

What Changed From the Old Breakdown (and why)

Change	Reason
Evaluation moved Phase 5 -> Phase 2	Discovering a broken protocol in the last phase leaves no time to fix it

Change	Reason
Single-LLM baseline moved into Phase 1	The core claim is comparative; the target must exist first
<code>as_of</code> on every data tool	Without it the backtest leaks the future
Post-cutoff test window + recall probe	Within-training-window backtesting measures memory, not forecasting
NewsAPI -> GDELT	NewsAPI free tier can't do historical/global; GDELT can, free
FRED key added to Phase 1	It was needed in Phase 2 but never provisioned
Two-class cache (permanent history)	A 15-min TTL re-fetches constantly and self-inflicts 429s
<code>--no-debate</code> ablation switch	The only way to measure if the debate earns its cost
Reconciliation gate + citations	Verified industry practice against numeric hallucination
Distinct state keys / reducer	Parallel LangGraph writes crash without this
Gemini schema gotchas surfaced	<code>Field(default=...)</code> rejected; nested schemas misread as tools
<code>pandas_ta_classic</code> import + <code>__init__.py</code>	The old test file fails forever without these
Security section added	Prompt injection, auth, key leaks — asked in any AI-agent viva
Dashboard demoted to Streamlit, last	High effort, low marks vs the evaluation work

References

- TradingAgents: Multi-Agents LLM Financial Trading Framework — [arXiv:2412.20138](#)
- Detecting Lookahead Bias in LLM Forecasts — [arXiv:2512.23847](#)
- Mitigating Look-Ahead Bias in Financial Backtesting with LLMs — [arXiv:2605.24564](#)
- LangGraph `INVALID_CONCURRENT_GRAPH_UPDATE` — LangChain docs
- Gemini structured output — googleapis/python-genai #699; pydantic-ai #3483
- GDELT Project — [gdeltproject.org](#) · yfinance exchange suffixes — [help.yahoo.com](#)